

OPERATING SYSTEMS — COMPLETE NOTES

Covers: College Exams | Placement Prep | Technical Interviews | Quick Revision

UNIT 1 — INTRODUCTION TO OPERATING SYSTEMS

1. Operating System Basics

Definition

An **Operating System (OS)** is system software that acts as an **intermediary between the user and the computer hardware**, managing hardware resources and providing services for application programs.

Need for Operating System

- **Hardware Abstraction** → Hides complex hardware details from users.
- **Resource Management** → Manages CPU, memory, I/O, and storage efficiently.
- **User Interface** → Provides CLI or GUI for user interaction.
- **Security** → Controls access to system resources.
- **Error Handling** → Detects and responds to hardware/software errors.

Goals of Operating System

- **Primary Goal** → Convenience — make computer easy to use.
- **Secondary Goal** → Efficiency — use hardware resources efficiently.
- **Tertiary Goal** → Ability to evolve — support new hardware and features.

Functions of Operating System

- **Process Management** → Create, schedule, terminate processes.
- **Memory Management** → Allocate and deallocate memory.
- **File-System Management** → Manage files, directories, and storage.
- **I/O Management** → Control I/O devices and drivers.

- **Security and Protection** → Control access to resources.
- **Networking** → Support communication between systems.
- **Error Detection** → Detect and handle hardware/software errors.
- **Accounting** → Track resource usage per user/process.

Evolution of Operating Systems

Era	Type	Key Feature
1940s–50s	No OS	Manual operation, punch cards
1950s–60s	Batch OS	Jobs batched and run sequentially
1960s	Multiprogramming	Multiple jobs in memory
1970s	Time-sharing	Interactive, multiple users
1980s	Personal Computer OS	Single-user, GUI (DOS, early Mac)
1990s	Network OS, Distributed OS	Multi-system communication
2000s+	Mobile, Cloud, Virtual	Android, iOS, VMs, containers

Types of Operating Systems

- **Batch OS** → Jobs collected in batches; no user interaction during execution. Example: IBM OS/360.
- **Multiprogramming OS** → Multiple jobs loaded in memory; CPU switches between them to reduce idle time.

- **Multitasking OS** → Extension of multiprogramming; fast CPU switching gives illusion of parallel execution. Example: UNIX.
- **Time-sharing OS** → CPU time is divided among multiple users; each gets a time slice. Example: MULTICS.
- **Distributed OS** → Multiple CPUs connected over a network; users see a single unified system. Example: Amoeba.
- **Network OS** → Computers connected via a network; each has its own OS but can share files and resources. Example: Novell NetWare.
- **Real-time OS (RTOS)** → Guarantees task completion within a strict time deadline. Used in embedded systems, aerospace. Example: VxWorks.
- **Embedded OS** → Designed for specific devices with limited hardware. Example: Embedded Linux, FreeRTOS.
- **Mobile OS** → Optimized for touchscreen, battery, and mobile hardware. Example: Android, iOS.

In One Line

An OS is the master program that manages hardware and provides an environment for other programs to run.

2. Computer System Organization

Computer System Components

Computer System

- ■ ■ CPU (Central Processing Unit)
- ■ ■ Main Memory (RAM)
- ■ ■ I/O Devices (keyboard, monitor, disk)
- ■ ■ Storage Devices (HDD, SSD)
- ■ ■ System Bus (connects all components)

CPU

- The **brain of the computer**; executes instructions.

- Components: **ALU** (arithmetic/logic), **Control Unit** (directs operations), **Registers** (fast temporary storage), **Cache** (fast memory close to CPU).

Main Memory

- **RAM (Random Access Memory)** — volatile, directly accessible by CPU.
- Instructions and data must be in main memory for execution.
- Faster than secondary storage but smaller in capacity.

I/O Devices

- Devices that allow input/output: keyboard, mouse, monitor, printer.
- Each I/O device has a **device controller** and a **device driver** in the OS.

Storage Devices

- **Primary Storage** → RAM (fast, volatile).
- **Secondary Storage** → HDD, SSD (slow, non-volatile, large capacity).
- **Tertiary Storage** → Tapes, optical discs (backup, archival).

System Bus

- **Data Bus** → Transfers actual data between components.
- **Address Bus** → Carries memory addresses.
- **Control Bus** → Carries control signals (read, write, interrupt).

Interrupts

- A **signal sent to the CPU** to indicate that an event needs immediate attention.
- **Hardware Interrupt** → Generated by hardware devices (e.g., keyboard press).
- **Software Interrupt** → Generated by programs (system calls, exceptions).
- **Interrupt Handler (ISR)** → Routine that executes when an interrupt occurs.

Device generates interrupt



CPU saves current state (registers, PC)



CPU jumps to Interrupt Handler (ISR)



ISR handles the interrupt



CPU restores state and resumes

DMA (Direct Memory Access)

- Allows I/O devices to **transfer data directly to/from memory** without involving the CPU for each byte.
- **Frees the CPU** to do other work during large data transfers.
- Used for disk I/O, network transfers, etc.
- After transfer is complete, DMA controller sends an **interrupt** to the CPU.

In One Line

A computer system is organized around the CPU, memory, I/O devices, and a system bus that connects them all.

3. Operating System Structure

OS Components

- **Kernel** → Core part of the OS; always running; manages hardware directly.
- **Shell** → Interface between user and kernel (CLI or GUI).
- **System Programs** → Utilities provided with the OS (compilers, editors).
- **Application Programs** → User programs (browsers, games).

Kernel

- The **core of the OS** that has full access to hardware.
- Manages processes, memory, I/O, and security.
- Runs in **kernel mode** (privileged mode).

User Mode vs Kernel Mode (Dual-Mode Operation)

Feature	User Mode	Kernel Mode
Privilege	Limited	Full
Hardware access	No direct access	Direct access
Used by	User applications	OS kernel
Protection	Cannot crash system	Can crash system
Switch trigger	System call / interrupt	Completion of system call

- **Dual-mode** protects the OS from faulty or malicious user programs.
- Mode bit = **1** → User mode; Mode bit = **0** → Kernel mode.

OS Architecture

Monolithic Structure

- **All OS services** run in a single large kernel.
- Fast but **difficult to maintain and debug**.
- Example: Early UNIX, MS-DOS.



Layered Structure

- OS divided into **layers**, each built on top of the lower layer.
- Layer 0 = Hardware; Layer N = User interface.

- Easy to debug but **poor performance** due to overhead.

Microkernel

- **Minimal kernel** — only essential services (IPC, basic scheduling, memory management).
- Other services run as **user-space processes**.
- **More secure and reliable** but slower due to message passing overhead.
- Example: Mach, MINIX, L4.

```
User Space:  [File Server] [Device Driver] [Network Server]
              ↑ Message Passing
Kernel Space: [Microkernel – IPC, Basic Scheduling, Memory]
Hardware
```

Modular Structure

- Kernel has a **set of core components** and can load additional modules dynamically.
- Best of monolithic and microkernel.
- Example: Linux (loadable kernel modules).

Hybrid Systems

- Combine aspects of multiple architectures.
- Example: Windows NT (monolithic + microkernel elements), macOS (Mach microkernel + BSD Unix layer).

In One Line

OS structure defines how kernel services are organized — from monolithic (all-in-one) to microkernel (minimal core).

4. Operating System Services

Service	Description
---------	-------------

Program Execution	Load and run programs; report errors on failure
I/O Operations	Manage I/O device access for programs
File-System Manipulation	Create, read, write, delete files and directories
Communication	Process-to-process communication (shared memory / message passing)
Error Detection	Detect and handle CPU, memory, and I/O errors
Resource Allocation	Allocate CPU, memory, and I/O devices among competing processes
Accounting	Track resource usage per user or process
Protection and Security	Control access; prevent unauthorized use

In One Line

OS services are the set of functions provided to users and programs to make computing safe, efficient, and convenient.

5. User–OS Interface

Command Line Interface (CLI)

- User types **text commands** directly.
- Commands interpreted by a **shell**.
- Faster for experts; scriptable and powerful.

- Example: Bash (Linux), CMD (Windows), PowerShell.

Graphical User Interface (GUI)

- User interacts via **windows, icons, menus, pointers (WIMP)**.
- More intuitive and user-friendly.
- Example: Windows Explorer, macOS Finder, GNOME.

Shell

- A **command interpreter** that reads user commands and executes them.
- Acts as the interface between user and OS kernel.
- Types: **sh, bash, zsh, csh** (Linux); **cmd, PowerShell** (Windows).

System Programs

- Programs provided **with the OS** to perform common tasks.
- Categories: File management, Status information, File modification, Language support, Communication, Application programs.
- Example: `ls`, `cp`, `grep`, `gcc`, `ping`.

In One Line

Users interact with the OS through CLI (text commands) or GUI (graphical interface), with the shell acting as the interpreter.

6. System Calls

System Call Concept

- A **system call** is a programmatic way for a user application to **request a service from the OS kernel**.
- Switches the process from **user mode to kernel mode**.
- After the service is done, returns to user mode.

System Call Interface

User Program

↓ (calls library function e.g., `printf()`)

C Library (wrapper function)

↓ (triggers system call via trap instruction)

Kernel (system call handler executes)

↓

Returns result to user program

Types of System Calls

Process Control

- `fork()` — Create a new process.
- `exec()` — Replace process image with a new program.
- `exit()` — Terminate a process.
- `wait()` — Wait for child process to finish.
- `getpid()` — Get process ID.

File Management

- `open()`, `close()` — Open/close a file.
- `read()`, `write()` — Read/write data.
- `create()`, `delete()` — Create/delete a file.

Device Management

- `ioctl()` — Device-specific I/O control.
- `read()`, `write()` — I/O on devices.

Information Maintenance

- `getpid()`, `getuid()` — Get process/user info.
- `time()`, `date()` — Get system time.
- `alarm()` — Set a timer.

Communication

- `pipe()` — Create a pipe.
- `socket()`, `bind()`, `connect()` — Network communication.
- `shmget()`, `shmat()` — Shared memory.

Protection

- `chmod()` — Change file permissions.
- `chown()` — Change file ownership.
- `umask()` — Set default permissions.

In One Line

System calls are the controlled entry points into the OS kernel, allowing user programs to request privileged services.

7. System Boot

Boot Process

```
Power ON
↓
BIOS/UEFI executes (stored in ROM)
↓
POST – Power-On Self-Test (hardware check)
↓
BIOS finds boot device (disk, USB, network)
↓
Bootloader loaded (MBR/GPT → first 512 bytes)
↓
Bootloader loads OS Kernel into memory
↓
Kernel initializes hardware drivers, memory, processes
↓
init / systemd starts user-space services
```



Login screen / Shell presented to user

Key Terms

- **BIOS (Basic Input Output System)** → Firmware in ROM; performs POST and finds bootloader. Old standard.
- **UEFI (Unified Extensible Firmware Interface)** → Modern replacement for BIOS; supports large disks, secure boot.
- **Bootloader** → Small program that loads the OS kernel. Example: GRUB (Linux), Windows Boot Manager.
- **Kernel Loading** → Kernel is decompressed and loaded into RAM; takes control of hardware.
- **OS Initialization** → Kernel sets up memory, creates first process (init/systemd), mounts root filesystem.

In One Line

Booting is the process of loading and starting the OS from power-on through firmware, bootloader, and kernel initialization.

8. Virtualization

Virtual Machines

- **Virtual Machine (VM)** → A software emulation of a physical computer that runs its own OS and applications.
- Multiple VMs can run on a **single physical machine**.
- Each VM is **isolated** from others.

Hypervisors

- **Hypervisor (Virtual Machine Monitor / VMM)** → Software that creates and manages VMs; sits between VMs and hardware.

Type 1 Hypervisor (Bare-Metal)

- Runs **directly on hardware**; no host OS.
- More efficient and secure.
- Example: VMware ESXi, Microsoft Hyper-V, Xen.

```
[VM1 (Guest OS)] [VM2 (Guest OS)] [VM3 (Guest OS)]  
      [Type 1 Hypervisor]  
      [Hardware]
```

Type 2 Hypervisor (Hosted)

- Runs **on top of a host OS** like an application.
- Easier to set up; less efficient.
- Example: VMware Workstation, VirtualBox.

```
[VM1 (Guest OS)] [VM2 (Guest OS)]  
      [Type 2 Hypervisor]  
      [Host OS]  
      [Hardware]
```

Virtualization Benefits

- **Isolation** → VMs don't affect each other.
- **Portability** → VMs can be moved between physical hosts.
- **Efficiency** → Multiple VMs on one server.
- **Testing** → Safe environment for experiments.
- **Cloud Computing** → Foundation of cloud infrastructure (AWS, Azure).

In One Line

Virtualization allows multiple OS instances (VMs) to run on a single physical machine, managed by a hypervisor.

UNIT 2 — PROCESS MANAGEMENT

1. Process Concept

Definition

A **process** is a **program in execution**. It is an active entity that includes the program code, current activity (PC, registers), stack, heap, and data section.

Program vs Process

Feature	Program	Process
Nature	Passive (stored on disk)	Active (running in memory)
Existence	Permanent	Temporary
Resources	None	Has CPU, memory, files
Multiple instances	One copy	Multiple processes possible

Process States



- **New** → Process is being created.

- **Ready** → Process is waiting for CPU assignment.
- **Running** → Instructions are being executed.
- **Waiting** → Process is waiting for I/O or an event.
- **Terminated** → Process has finished execution.

Process Control Block (PCB)

A **PCB** is a data structure maintained by the OS for each process, containing all information about the process.

Field	Description
Process ID (PID)	Unique identifier
Process State	New, Ready, Running, Waiting, Terminated
Program Counter	Address of next instruction
CPU Registers	All register values
CPU Scheduling Info	Priority, queue pointers
Memory Management Info	Page table, base/limit registers
I/O Status Info	Open files, allocated I/O devices
Accounting Info	CPU time used, time limits

Process Creation

- A process is created by another process (parent creates child).
- New process gets a unique PID.
- Child may **share resources** with parent or get its own.

Process Termination

- Process calls `exit()` → OS deallocates all resources.
- Parent may wait for child via `wait()`.
- **Cascading termination** → If parent terminates, OS may terminate all children.

In One Line

A process is a program in execution, described by its PCB, and moves through states: New → Ready → Running → Waiting → Terminated.

2. Process Operations

Parent and Child Processes

- The **creating process** is the parent; the **created process** is the child.
- Forms a **process tree** (hierarchy).
- In UNIX, the first process is `init` (PID=1) — parent of all processes.

Key System Calls

`fork()`

- Creates a **new child process** that is an exact copy of the parent.
- Returns **PID of child to parent**, returns **0 to child**.
- Parent and child execute **concurrently** after fork.

Parent Process

↓ `fork()`

■■■■■■■■■■

Parent Child (copy of parent)

(gets child PID) (gets 0)

`exec()`

- **Replaces the current process image** with a new program.

- Does not create a new process; reuses the same PID.
- Used after `fork()` to run a different program in the child.

`wait()`

- Parent **waits for child process to finish**.
- After child terminates, parent is unblocked and gets child's exit status.

`exit()`

- Process **terminates itself** voluntarily.
- All resources are released; parent is notified.

Zombie Process

- A process that has **completed execution** but its **PCB still exists** in the process table because the parent hasn't called `wait()`.
- Entry remains so parent can read the exit status.
- **Risk:** Too many zombies exhaust the process table.

Orphan Process

- A process whose **parent has terminated** before it does.
- In UNIX, orphans are **adopted by `init`** (or `systemd`), which calls `wait()` for them.

In One Line

Processes are created with `fork()`, replaced with `exec()`, and cleaned up with `wait()` and `exit()`; zombies/orphans arise from improper parent-child lifecycle management.

3. Process Scheduling

Scheduling Concept

- **CPU scheduling** selects which process in the Ready queue gets the CPU next.
- Goal: Maximize CPU utilization and throughput; minimize waiting time.

Types of Schedulers

Scheduler	Also Called	Job	Frequency
Long-term	Job Scheduler	Selects jobs from job pool → brings into Ready queue	Infrequent
Short-term	CPU Scheduler	Selects from Ready queue → gives CPU	Very frequent (ms)
Medium-term	Swapper	Swaps processes in/out of memory	Moderate

Dispatcher

- The **dispatcher** gives actual control of CPU to the selected process.
- Responsibilities: **Context switching**, switching to user mode, jumping to correct location in the program.
- **Dispatch latency** = Time taken by dispatcher to stop one process and start another.

Context Switching

- When CPU switches from one process to another, the **state of the old process is saved** (into its PCB) and the **state of the new process is restored** (from its PCB).
- Pure **overhead** — no useful work done during context switch.
- Time depends on hardware support (register count, memory speed).

```
Process P0 running
  ↓ interrupt/system call
Save P0 state into PCB0
Load P1 state from PCB1
  ↓
Process P1 running
```

↓ interrupt

Save P1 state into PCB1

Load P0 state from PCB0

↓

Process P0 resumes

In One Line

Scheduling determines which process gets the CPU; the dispatcher performs the actual context switch.

4. Scheduling Criteria

Criterion	Goal	Description
CPU Utilization	Maximize	Keep CPU as busy as possible (ideal: 40–90%)
Throughput	Maximize	Number of processes completed per unit time
Turnaround Time	Minimize	Total time from submission to completion
Waiting Time	Minimize	Total time spent in Ready queue
Response Time	Minimize	Time from request to first response (important for interactive systems)

Formulas

Turnaround Time (TAT) = Completion Time – Arrival Time
 Waiting Time (WT) = Turnaround Time – Burst Time
 Response Time (RT) = First CPU allocation time – Arrival Time
 Average WT = Sum of all Waiting Times / Number of processes
 Average TAT = Sum of all Turnaround Times / Number of processes

5. CPU Scheduling Algorithms

1. FCFS — First Come First Served

- **Non-preemptive** — process that arrives first gets CPU first.
- Simple; no starvation.
- **Convoy effect** → Short jobs wait behind long ones; high average waiting time.

Example:

Process	Arrival	Burst
P1	0	5
P2	1	3
P3	2	2

Gantt Chart: | P1(0-5) | P2(5-8) | P3(8-10) |

Process	CT	TAT	WT
P1	5	5	0
P2	8	7	4
P3	10	8	6

Avg WT = $(0+4+6)/3 = 3.33$ | Avg TAT = $(5+7+8)/3 = 6.67$

2. SJF — Shortest Job First

- **Non-preemptive** — selects the process with **shortest burst time** in Ready queue.
 - **Optimal** average waiting time (non-preemptive version).
 - **Problem:** Starvation of long processes; burst time must be known in advance.
-

3. SRTF — Shortest Remaining Time First

- **Preemptive version of SJF.**
 - If a new process arrives with a shorter remaining time than the current process, CPU is preempted.
 - **Optimal** average waiting time overall.
-

4. Priority Scheduling

- Each process has a **priority number**; CPU given to highest priority (lower number = higher priority in most systems).
 - Can be **preemptive** or **non-preemptive**.
 - **Problem:** Starvation of low-priority processes.
 - **Solution:** Aging — gradually increase priority of waiting processes.
-

5. Round Robin (RR)

- Each process gets CPU for a fixed **time quantum (q)**.
- After quantum expires, process is **preempted** and moved to the end of Ready queue.
- **Preemptive**; designed for time-sharing systems.
- **Performance depends on quantum size:**
- Large q → behaves like FCFS.
- Small q → lots of context switches → high overhead.
- No starvation; fair allocation.

6. Multilevel Queue Scheduling

- Ready queue is divided into **multiple queues** (e.g., foreground, background).
- Each queue has its own scheduling algorithm.
- **Fixed priority** between queues — processes cannot move between queues.
- Example: Foreground (interactive) → RR; Background (batch) → FCFS.

7. Multilevel Feedback Queue

- Like multilevel queue, but processes **can move between queues** based on behavior.
- CPU-bound processes move to **lower-priority queues**.
- I/O-bound or short processes stay in **higher-priority queues**.
- Most flexible and complex; used in most modern OSes.

Numerical Problem Template

Given: Arrival Time (AT), Burst Time (BT) for each process.

Steps: 1. Draw the Gantt chart. 2. Find **Completion Time (CT)** for each process. 3. **TAT = CT – AT** 4. **WT = TAT – BT** 5. Calculate **Avg WT** and **Avg TAT**.

In One Line

FCFS is simple but slow; SJF gives optimal WT; RR is fair; Priority needs aging to prevent starvation; Multilevel Feedback Queue is the most practical.

6. Threads

Thread Concept

- A **thread** is the smallest unit of CPU execution within a process.
- A process can have **multiple threads** that share the same code, data, and files but have their own **stack, registers, and program counter**.

Process vs Thread

Feature	Process	Thread
Definition	Program in execution	Segment of a process
Memory	Separate address space	Shared address space
Creation overhead	High	Low
Communication	IPC required	Direct shared memory
Context switch cost	High	Low
Failure impact	Does not affect others	Can crash entire process

Benefits of Threads

- **Responsiveness** → UI stays responsive while background tasks run.
- **Resource Sharing** → Threads share memory; no extra IPC needed.
- **Economy** → Cheaper to create/context-switch than processes.
- **Scalability** → Can utilize multiple CPU cores.

User-Level Threads (ULT)

- Managed by **user-space thread library** (not the kernel).
- Kernel sees only one process; doesn't know about threads.
- Fast to create/switch; **one thread blocked** → **all blocked**.
- Example: POSIX pthreads (user level), Java threads (early versions).

Kernel-Level Threads (KLT)

- Managed directly by the **OS kernel**.
- Kernel knows about each thread; can schedule them on different CPUs.

- Slower to create/switch; **one thread blocked** → **others continue**.
- Example: Windows threads, Linux tasks.

Multithreading Models

Many-to-One

```
[Thread][Thread][Thread] ← User threads
      ↓
[Kernel Thread] ← One kernel thread
```

- Multiple user threads → one kernel thread.
- **Blocking one blocks all**; cannot run in parallel.

One-to-One

```
[Thread1] → [KT1]
[Thread2] → [KT2] ← Each user thread maps to a kernel thread
[Thread3] → [KT3]
```

- Most flexible; true parallelism; used by Linux and Windows.
- Creating many user threads creates many kernel threads → overhead.

Many-to-Many

```
[T1][T2][T3][T4] ← User threads
      ⇕
[KT1][KT2] ← Kernel threads
```

- Best of both worlds; OS creates enough kernel threads.
- Complex to implement.

In One Line

Threads are lightweight units of execution within a process, sharing resources but with independent control flow.

7. Inter-Process Communication (IPC)

IPC Concept

- **IPC** allows processes to **communicate and synchronize** with each other.
- Two main models: **Shared Memory** and **Message Passing**.

Shared Memory

- A **region of memory** shared between two or more processes.
- Fastest IPC — no kernel involvement after setup.
- Requires **synchronization** to avoid race conditions.
- Example: `shmget()`, `shmat()` in UNIX.

Message Passing

- Processes communicate by **sending and receiving messages** through the OS kernel.
- Easier to implement in distributed systems.
- Slower than shared memory (kernel involved in each message).
- Example: `send()`, `receive()` system calls.

Pipes

- A **unidirectional channel** for communication between related processes (parent-child).
- **Anonymous pipe** → only between related processes.
- **Named pipe (FIFO)** → can be used by unrelated processes.
- One end writes, the other reads.

Sockets

- **Endpoint** for communication over a network (or locally).
- Used for **client-server** communication.
- Identified by IP address + port number.
- Support both TCP (reliable) and UDP (unreliable).

Signals

- **Software interrupts** sent to a process to notify it of an event.
- Example: `SIGKILL` (terminate), `SIGSEGV` (segmentation fault), `SIGTERM` (terminate gracefully).
- Process can **handle, ignore, or block** signals (except `SIGKILL` and `SIGSTOP`).

In One Line

IPC allows processes to exchange data using shared memory (fast) or message passing (safe), with pipes, sockets, and signals as common mechanisms.

UNIT 3 — PROCESS SYNCHRONIZATION AND DEADLOCKS

1. Process Synchronization

Race Condition

- When **multiple processes access and manipulate shared data concurrently**, and the outcome depends on the **order of access**.
- Result is incorrect or unpredictable.
- Example: Two processes simultaneously incrementing a shared counter.

Critical Section

- A **segment of code** where a process accesses **shared resources** (shared variables, files, hardware).
- Only **one process** should be in its critical section at a time.

Critical-Section Problem

Given N processes, design a protocol so that: - **Only one process executes** in its critical section at a time.

Requirements for a Valid Solution

1. **Mutual Exclusion** → Only one process in the critical section at a time.
2. **Progress** → If no process is in the CS, selection of the next process to enter cannot be postponed indefinitely.
3. **Bounded Waiting** → A limit must exist on how many times other processes can enter CS before a waiting process gets its turn (no starvation).

In One Line

The critical-section problem requires mutual exclusion, progress, and bounded waiting to ensure correct access to shared resources.

2. Synchronization Solutions

Peterson's Solution

- **Software solution** for two processes.
- Uses two shared variables: `flag[]` (intent) and `turn` (whose turn).

```
// Process Pi
flag[i] = true;
turn = j;
while (flag[j] && turn == j);
// CRITICAL SECTION
flag[i] = false;
// REMAINDER SECTION
```

- Satisfies all three requirements for 2 processes.
- **Not guaranteed to work** on modern CPU architectures (due to instruction reordering).

Hardware Synchronization

Modern CPUs provide **atomic instructions** to solve the critical-section problem.

Atomic Operations

- Operations that **execute completely without interruption**.
- No other process can observe a partial state.

Test-and-Set (TAS)

```
boolean test_and_set(boolean *target) {  
    boolean rv = *target;  
    *target = true;  
    return rv;  
}  
// Usage (lock = false initially):  
while (test_and_set(&lock)); // acquire lock  
// Critical Section  
lock = false;                // release lock
```

Compare-and-Swap (CAS)

```
int compare_and_swap(int *value, int expected, int new_value) {  
    int temp = *value;  
    if (*value == expected) *value = new_value;  
    return temp;  
}
```

- Used in **lock-free** data structures.
- Both TAS and CAS satisfy **mutual exclusion** but may not satisfy **bounded waiting** by themselves.

In One Line

Peterson's is a software solution; hardware solutions like TAS and CAS provide atomic operations for correct synchronization.

3. Semaphores

Semaphore Concept

- A **semaphore S** is an integer variable accessed only through two atomic operations: **wait()** and **signal()**.
- Used for both **mutual exclusion** and **process synchronization**.

wait() and signal()

```
wait(S):  
    while (S <= 0); // busy wait  
    S--;  
  
signal(S):  
    S++;
```

Binary Semaphore (Mutex)

- Value is either **0** or **1**.
- Used to achieve **mutual exclusion** (acts like a lock).
- Only one process in critical section at a time.

Counting Semaphore

- Value can be **any non-negative integer**.
- Used to control access to a resource with **multiple instances**.
- S = number of available resources.

Blocking Implementation (No Busy-Wait)

```
wait(S):  
    S.value--;  
    if (S.value < 0):
```

```

        add process to S.list
        block()
    }
signal(S):
    S.value++;
    if (S.value <= 0):
        remove process P from S.list
        wakeup(P)

```

In One Line

Semaphores are integer synchronization primitives; binary semaphores provide mutual exclusion, counting semaphores manage resource pools.

4. Classical Synchronization Problems

Producer–Consumer Problem (Bounded Buffer)

- **Producer** generates data; **Consumer** consumes it; shared **bounded buffer** of size N.
- Problem: Producer must not add to full buffer; Consumer must not remove from empty buffer.
- **Solution:** Three semaphores: `mutex=1` (mutual exclusion), `full=0` (count of full slots), `empty=N` (count of empty slots).

```

// Producer
wait(empty);
wait(mutex);
// add item to buffer
signal(mutex);
signal(full);

// Consumer
wait(full);
wait(mutex);
// remove item from buffer

```

```
signal(mutex);  
signal(empty);
```

Readers–Writers Problem

- Multiple **readers** can read simultaneously; **writers** need exclusive access.
- **Problem:** Starvation of writers if readers keep arriving.
- **Solution:** `mutex` semaphore for reader count, `rw_mutex` for writer access.
- `rw_mutex` is locked by the **first reader** and unlocked by the **last reader**.

Dining Philosophers Problem

- **5 philosophers** sit around a table; 5 chopsticks between them.
- Each philosopher needs **2 chopsticks** to eat; only picks up adjacent ones.
- **Problem:** Deadlock (all pick left chopstick simultaneously; none can pick right).
- **Solutions:**
 - Allow at most **4 philosophers** to sit simultaneously.
 - **Odd** philosophers pick left first; **even** pick right first.
 - Use a **monitor** to check both chopsticks are available before picking.

In One Line

Classical problems (Producer-Consumer, Readers-Writers, Dining Philosophers) illustrate common synchronization challenges and their semaphore-based solutions.

5. Monitors

Monitor Concept

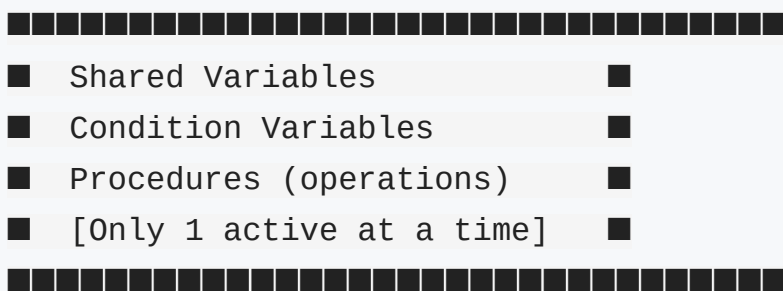
- A **high-level synchronization construct** that bundles shared data, operations, and synchronization into one module.
- **Only one process** can be active within the monitor at a time (mutual exclusion is automatic).

- Simpler to use correctly than semaphores.

Condition Variables

- Used inside monitors to allow a process to **wait** for a condition.
- Two operations:
 - `x.wait()` → Process suspends itself; releases monitor lock.
 - `x.signal()` → Resumes one suspended process; if none waiting, has no effect.

Monitor Structure:



Monitor-Based Synchronization

- Dining Philosophers can be elegantly solved with a monitor.
- Monitor ensures a philosopher picks up both chopsticks **atomically** or waits.

In One Line

Monitors are high-level synchronization structures that automatically enforce mutual exclusion and provide condition variables for waiting.

6. Deadlocks

Deadlock Concept

- A **deadlock** is a situation where a **set of processes are permanently blocked**, each waiting for a resource held by another process in the set.

Necessary Conditions (All Four Must Hold Simultaneously)

1. **Mutual Exclusion** → At least one resource must be held in a non-shareable mode.
2. **Hold and Wait** → A process holds at least one resource while waiting for additional resources.
3. **No Preemption** → Resources cannot be forcibly taken; must be released voluntarily.
4. **Circular Wait** → A circular chain of processes exists, each waiting for a resource held by the next.

In One Line

Deadlock occurs when four conditions hold simultaneously: mutual exclusion, hold-and-wait, no preemption, and circular wait.

7. Deadlock Handling

Deadlock Prevention

- **Eliminate at least one** of the four necessary conditions.

Condition	Prevention Strategy
Mutual Exclusion	Make resources shareable (not always possible)
Hold and Wait	Request all resources at once before starting
No Preemption	Allow OS to preempt resources from waiting processes
Circular Wait	Impose a total ordering on resource types; request in order

- **Problem:** Low resource utilization; possible starvation.

Deadlock Avoidance

- OS is given **advance information** about resource needs.
- OS grants resources only if the system remains in a **safe state**.
- **Safe State** → There exists a sequence in which every process can finish.
- Uses **Banker's Algorithm**.

Deadlock Detection

- Allow deadlocks to occur; periodically run a **detection algorithm**.
- Uses **Resource Allocation Graph** or **detection algorithm**.
- Recovery needed after detection.

Deadlock Recovery

1. **Process Termination:** - Abort all deadlocked processes (drastic). - Abort one process at a time until deadlock breaks.
2. **Resource Preemption:** - Select a **victim** process; take its resources. - **Rollback** victim to safe state. - Prevent **starvation** (don't always pick same victim).

In One Line

Deadlocks can be handled by prevention (eliminate conditions), avoidance (Banker's Algorithm), detection+recovery, or simply ignoring them (Ostrich algorithm).

8. Deadlock Algorithms

Resource Allocation Graph (RAG)

- **Nodes:** Processes (circles) and Resources (squares).
- **Request Edge:** $P \rightarrow R$ (process requests resource).
- **Assignment Edge:** $R \rightarrow P$ (resource assigned to process).
- **Deadlock detected** if graph contains a **cycle** (for single-instance resources).

$P1 \rightarrow R1 \rightarrow P2 \rightarrow R2 \rightarrow P1 \leftarrow \text{Cycle} = \text{Deadlock}$

Banker's Algorithm

Used for **deadlock avoidance** with multiple instances of resources.

Data Structures (for n processes, m resource types): - `Available[m]` → Available instances of each resource type. - `Max[n][m]` → Maximum demand of each process. - `Allocation[n][m]` → Resources currently allocated to each process. - `Need[n][m]` → Remaining need = `Max - Allocation`.

Safety Algorithm

1. `Work = Available`, `Finish[i] = false` for all i
2. Find process P_i such that:
 - `Finish[i] == false`
 - `Need[i] <= Work`
3. `Work = Work + Allocation[i]`, `Finish[i] = true`
4. Repeat until no such P_i found
5. If all `Finish[i] == true` → Safe State; otherwise Unsafe

Resource-Request Algorithm

- When process P_i requests resources `Request[i]`:
1. If `Request[i] <= Need[i]`, proceed; else error (exceeded max claim)
 2. If `Request[i] <= Available`, proceed; else P_i waits
 3. Pretend to allocate:
 - `Available = Available - Request[i]`
 - `Allocation[i] = Allocation[i] + Request[i]`
 - `Need[i] = Need[i] - Request[i]`
 4. Run Safety Algorithm
 - If safe → grant allocation
 - If unsafe → restore old state, P_i waits

Deadlock Detection Algorithm (Multiple Instances)

Similar to Banker's safety algorithm but uses `Allocation` and `Request` (current requests, not max).

1. $Work = Available$, $Finish[i] = (Allocation[i] == 0)$
2. Find P_i : $Finish[i] = false$ AND $Request[i] \leq Work$
3. $Work = Work + Allocation[i]$, $Finish[i] = true$
4. If any $Finish[i] == false \rightarrow$ those processes are deadlocked

Banker's Algorithm Numerical Template

Given: Available, Max, Allocation $\rightarrow$ Compute Need = Max – Allocation.

Example:

Process	Allocation (A B C)	Max (A B C)	Need (A B C)
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1
P4	0 0 2	4 3 3	4 3 1

Available = (3, 3, 2)

Safe Sequence: P1 $\rightarrow$ P3 $\rightarrow$ P4 $\rightarrow$ P0 $\rightarrow$ P2

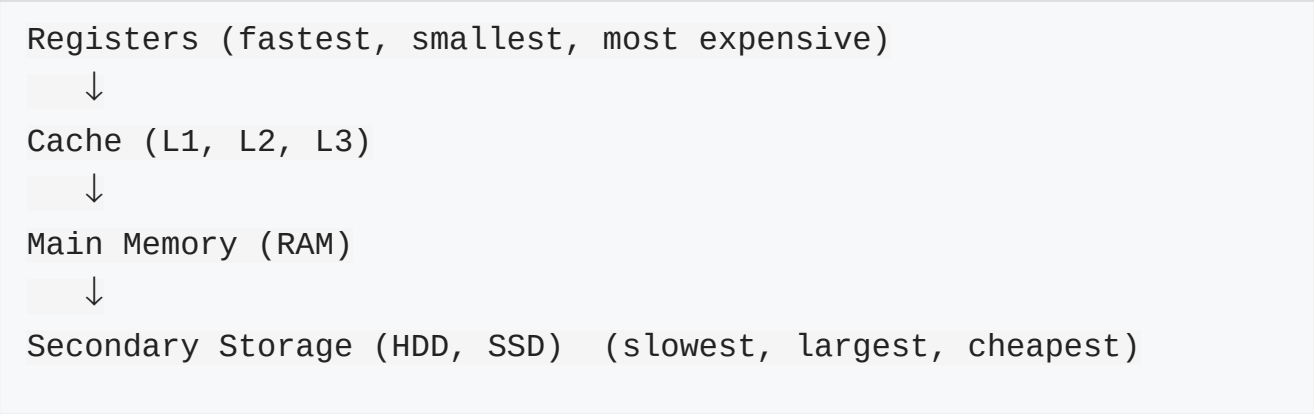
In One Line

Banker's Algorithm avoids deadlock by granting resources only when a safe execution sequence exists; RAG detects deadlock through cycles.

UNIT 4 — MEMORY MANAGEMENT

1. Memory Management Basics

Memory Hierarchy



Address Binding

Converting a logical (program) address to a physical (memory) address.

- **Compile Time** → Absolute code; physical address hardcoded (must reload if changed).
- **Load Time** → Relocatable code; binding at load; fixed once loaded.
- **Execution Time** → Binding delayed until runtime; allows process to move during execution; needs hardware support (MMU).

Logical vs Physical Address

Feature	Logical Address	Physical Address
Also called	Virtual address	Real address
Generated by	CPU	MMU
Seen by	User/Process	Hardware/Memory

Range	Logical address space	Physical address space
-------	-----------------------	------------------------

MMU (Memory Management Unit)

- Hardware device that **maps logical addresses to physical addresses** at runtime.
- Uses a **relocation register (base register)**: $\text{Physical} = \text{Logical} + \text{Base}$.

Dynamic Loading

- A routine is **loaded into memory only when it is called**.
- Unused routines are never loaded → better memory utilization.
- No special OS support needed; user's responsibility.

Dynamic Linking

- **Linking deferred until runtime**.
- Shared libraries (`.dll` / `.so`) are linked when the program is run.
- One copy of library in memory shared among all programs.
- Requires OS support to locate the library.

In One Line

Memory management involves address binding (compile/load/execution time), logical-to-physical address translation by MMU, and efficient memory use via dynamic loading/linking.

2. Contiguous Memory Allocation

Single Partition

- Memory divided into two regions: **OS** and **one user process**.
- Relocation register protects OS; limit register protects other processes.
- Simple but wastes memory.

Multiple Partitions

Fixed Partitioning

- Memory divided into **fixed-size partitions** at system start.
- One process per partition.
- **Simple** but suffers from **internal fragmentation** (partition bigger than process → wasted space inside partition).

Variable Partitioning

- **Partitions created dynamically** to fit each process exactly.
- No internal fragmentation, but **external fragmentation** builds up over time.
- Requires compaction to solve external fragmentation.

3. Allocation Techniques

When a process arrives and needs memory, the OS picks a **free hole** from the list:

Technique	Strategy	Pros	Cons
First Fit	Allocate first hole that is big enough	Fast	May leave large holes fragmented
Best Fit	Allocate smallest hole that fits	Minimizes wasted space	Slow; leaves tiny leftover holes
Worst Fit	Allocate largest hole	Leaves large leftovers	Slow; wastes large holes
Next Fit	Like First Fit but starts from last allocated	Distributes allocations	Similar to First Fit

First Fit and Best Fit are generally better than Worst Fit.

4. Fragmentation

Internal Fragmentation

- **Wasted space inside an allocated partition** (allocated block bigger than requested).
- Occurs in **fixed partitioning** and **paging**.

External Fragmentation

- **Enough total free memory exists, but it is scattered** in small non-contiguous holes.
- Occurs in **variable partitioning** and **segmentation**.

Compaction

- **Move all processes together** to one end of memory; combine all free holes into one large block.
- **Reduces external fragmentation**.
- Expensive: requires relocating all processes; only possible if relocation is dynamic.

5. Swapping

Swapping Concept

- A process can be **temporarily removed from main memory** (swapped out) to a **backing store** (swap space on disk) and later brought back (swapped in).
- Allows **more processes** than physical memory can hold to run (virtually).

Swap Space

- Dedicated area on disk for storing swapped-out processes.
- Must be fast; typically a separate disk partition.

Swapping Techniques

- **Standard Swapping** → Entire process moved to disk.

- **Swapping with Paging** → Only some pages swapped (more common today — forms basis of virtual memory).
 - Modern OS uses **lazy swapping**: swap out only when necessary.
-

6. Paging

Paging Concept

- Physical memory divided into fixed-size blocks called **frames**.
- Logical memory divided into same-size blocks called **pages**.
- **No external fragmentation** (frames allocated anywhere).
- Small **internal fragmentation** (last page may not be full).

Key Terms

- **Page** → Fixed-size block of logical address space.
- **Frame** → Fixed-size block of physical memory.
- **Page Table** → Maps page numbers to frame numbers.
- **Page Size** → Typically 4KB; power of 2.

Address Translation

```
Logical Address = [Page Number (p) | Page Offset (d)]
                ↓
                Page Table[p] = Frame Number (f)
                ↓
Physical Address = [Frame Number (f) | Page Offset (d)]
```

TLB (Translation Lookaside Buffer)

- A **fast hardware cache** for recent page table entries.
- If page number found in TLB → **TLB hit** (fast).
- If not found → **TLB miss** → access page table in memory (slow).

Effective Access Time (EAT):

$$\text{EAT} = (\text{TLB access time} + \text{Memory access time}) \times \text{Hit ratio} \\ + (\text{TLB access time} + 2 \times \text{Memory access time}) \times (1 - \text{Hit ratio})$$

Multilevel Paging

- For large address spaces, page table itself becomes large.
- Solution: **Divide page table into pages** (two-level, three-level paging).
- Example (32-bit, 4KB pages): Two-level paging = outer page table + inner page tables.

Inverted Page Table

- One entry per **physical frame** (not per logical page).
 - Reduces page table size.
 - Searching is slower (need to search whole table for a given PID + page number).
 - Used in IBM systems, PowerPC.
-

7. Segmentation

Segmentation Concept

- Divides logical memory into **variable-size segments** based on logical divisions of a program (code, data, stack, heap).
- More natural view of memory from programmer's perspective.
- **External fragmentation** possible (like variable partitioning).

Segment Table

- Each entry has **base** (starting physical address) and **limit** (size of segment).

Logical Address in Segmentation

Logical Address = [Segment Number (s) | Offset (d)]

↓

Segment Table[s] → Base + Limit

↓

If d < Limit: Physical = Base + d

Else: Segmentation Fault

Segmentation with Paging

- Combine segmentation and paging: each segment is paged.
- Gets benefits of both: logical view + no external fragmentation.
- Example: Intel x86 architecture.

8. Virtual Memory

Virtual Memory Concept

- Allows a program to run even if it is **not entirely in main memory**.
- Only the **needed parts** are loaded; rest stays on disk.
- Program can be **larger than physical memory**.

Demand Paging

- Pages are loaded into memory **only when needed** (on demand).
- If a page is needed and not in memory → **page fault** occurs.
- Uses a **valid-invalid bit** in page table: valid = in memory; invalid = not loaded.

Page Fault Handling

1. Process accesses a page
2. Page table: invalid bit set → Page Fault (trap to OS)
3. OS checks if reference is valid (legal address?)
4. If valid: find a free frame
5. Load required page from disk into frame

6. Update page table: set valid bit
7. Restart the instruction that caused the fault

Copy-on-Write (COW)

- After `fork()`, parent and child **share the same pages** (no immediate copy).
 - When either process **writes** to a shared page → OS creates a copy of that page.
 - Reduces memory usage and speeds up `fork()`.
-

9. Page Replacement Algorithms

When a page fault occurs and **no free frame is available**, the OS must **replace** an existing page.

FIFO (First-In First-Out)

- Replace the **oldest page** in memory.
- Simple but may replace heavily-used pages.
- **Bélády's Anomaly** → Increasing frames can increase page faults.

Optimal (OPT)

- Replace the page that will **not be used for the longest time** in the future.
- Theoretically optimal (minimum page faults).
- **Not implementable** in practice (requires future knowledge); used as benchmark.

LRU (Least Recently Used)

- Replace the page that **has not been used for the longest time** in the past.
- Good approximation of Optimal.
- Implementation: **Counter** (timestamp each access) or **Stack** (recently used at top).

Second Chance (Clock Algorithm)

- Modified FIFO with a **reference bit**.

- When a page is accessed, reference bit = 1.
- On replacement: if reference bit = 0 → replace; if = 1 → give second chance (set to 0, move on).
- **Clock algorithm** uses a circular queue with a clock hand.

Page Replacement Numerical Template

Reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2 | Frames = 3

Track which pages are in memory at each step; mark page faults (F).

Step	Pages in Memory	Fault?
7	7	F
0	7, 0	F
1	7, 0, 1	F
2	0, 1, 2 (replace 7)	F
...	...	...

TLB Numerical — Effective Access Time

Given: TLB access time = 20ns, Memory access time = 100ns, Hit ratio = 90%

$$\begin{aligned}
 \text{EAT} &= 0.9 \times (20 + 100) + 0.1 \times (20 + 100 + 100) \\
 &= 0.9 \times 120 + 0.1 \times 220 \\
 &= 108 + 22 \\
 &= 130 \text{ ns}
 \end{aligned}$$

10. Thrashing

Thrashing Concept

- A process is **thrashing** when it spends **more time paging than executing**.
- Occurs when a process does not have enough frames; causes frequent page faults.

Causes

- Too many processes in memory → each gets too few frames → high page fault rate.
- OS increases multiprogramming → CPU utilization drops → OS adds more processes → worse thrashing.

↑ Multiprogramming → ↓ Frames/Process → ↑ Page Faults → ↓ CPU Utilization

Working-Set Model

- **Working set** of a process = set of pages it is actively using in a recent window Δ .
- OS allocates **enough frames** to accommodate each process's working set.
- If sum of working sets > total frames → **suspend a process**.

Page-Fault Frequency (PFF)

- Monitor page-fault rate per process.
- If too **high** → give process more frames.
- If too **low** → take frames away.
- Keeps page-fault rate within an acceptable range.

Thrashing Control

- **Reduce degree of multiprogramming** → Swap out some processes.
- Use **working-set model** or **PFF** to allocate frames.
- Provide more physical memory (hardware solution).

In One Line

Thrashing happens when too little memory is given to a process causing excessive paging; controlled by the working-set model and page-fault

frequency.

UNIT 5 — FILE SYSTEM AND I/O MANAGEMENT

1. File System

File Concept

- A **file** is a named collection of related information stored on secondary storage.
- Smallest logical unit of storage from user's perspective.

File Attributes

Attribute	Description
Name	Human-readable identifier
Identifier	Unique tag (inode number)
Type	Text, binary, executable, etc.
Location	Pointer to file on device
Size	Current file size
Protection	Access control (read, write, execute)
Timestamps	Created, accessed, modified

File Operations

- **Create** → Allocate space; make directory entry.

- **Write** → Write at current write pointer.
- **Read** → Read at current read pointer.
- **Seek** → Move file pointer to given position.
- **Delete** → Remove directory entry; release space.
- **Truncate** → Keep attributes; erase content; reset size to 0.

File Types

Type	Extension	Description
Executable	.exe, .out	Ready-to-run machine code
Object	.o, .obj	Compiled, not yet linked
Source code	.c, .java	Human-readable source
Text	.txt, .md	Plain text
Archive	.zip, .tar	Packed files
Multimedia	.mp3, .mp4	Audio/video

File Access Methods

- **Sequential Access** → Read/write in order from beginning; most common. Example: tape.
 - **Direct (Random) Access** → Read/write at any block directly using block number. Example: disk.
 - **Indexed Access** → Use an index to find the block; faster search. Example: database index.
-

2. Directory Structure

Single-Level Directory

- All files in **one directory** for all users.
- **Problem:** Naming conflicts; no grouping.

Two-Level Directory

- **Separate directory for each user**; master file directory at top level.
- Solves naming conflicts between users.
- **Problem:** No grouping within user's directory.

Tree-Structured Directory

- **Hierarchical directory tree**; users can create subdirectories.
- **Absolute path** → From root to file. Example: `/home/user/docs/file.txt`
- **Relative path** → From current directory. Example: `docs/file.txt`
- Most common structure (Linux, Windows).

Acyclic Graph Directory

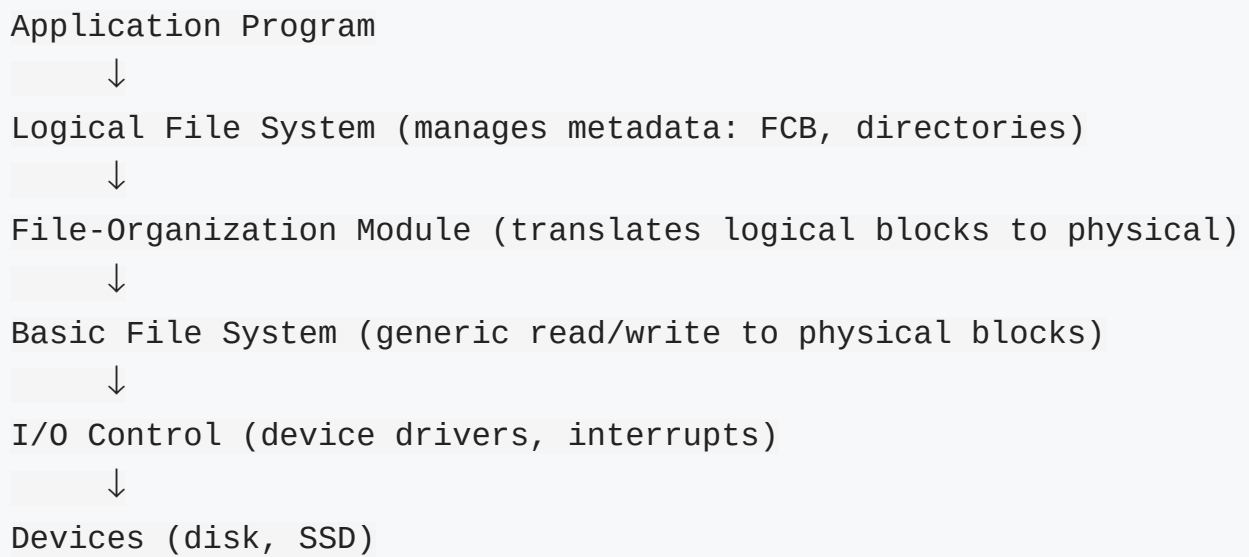
- Allows **sharing of files and subdirectories** (two names can refer to same file).
- Implemented using **hard links** or **symbolic links**.
- **Problem:** Dangling pointer if original deleted (for symbolic links).

General Graph Directory

- **Cycles allowed** in the directory graph.
- Most flexible but complex; requires **garbage collection** to reclaim space.
- Used rarely; cycles can cause infinite loops in traversal.

3. File-System Implementation

File-System Structure



File Control Block (FCB) / Inode

- A **data structure** containing all information about a file.
- Stored on disk; one FCB per file.
- Contains: permissions, dates, owner, size, location of data blocks.
- In UNIX: called **inode**.

File-System Mounting

- Before a file system can be accessed, it must be **mounted** at a **mount point**.
- Mount point = existing directory in the current file system.
- Example: `mount /dev/sdb1 /mnt/usb`

In-Memory File System Structures

- **Mount table** → Info about mounted volumes.
 - **Directory cache** → Recently accessed directories.
 - **System-wide open-file table** → One entry per open file.
 - **Per-process open-file table** → One entry per open file per process.
-

4. File Allocation Methods

Contiguous Allocation

- Each file occupies a **contiguous set of blocks** on disk.
- Simple; fast sequential and random access.
- **External fragmentation**; file size must be known in advance.

```
File A: Block 0,1,2  
File B: Block 5,6,7,8
```

Linked Allocation

- Each block contains a **pointer to the next block**.
- No external fragmentation; file can grow dynamically.
- **Poor random access** (must traverse from start); pointer overhead; reliable issues (lost pointer = lost data).

```
Block 0 → Block 5 → Block 11 → Block 3 → NULL
```

FAT (File Allocation Table) → Improvement: pointers stored in a table in memory (used by FAT32).

Indexed Allocation

- Each file has an **index block** containing pointers to all its data blocks.
- Fast random access; no external fragmentation.
- **Overhead** of index block; small files waste space on index.

```
Index Block: [3][7][12][2] → points to data blocks 3, 7, 12, 2
```

Multi-level index → For large files, index block points to other index blocks.

5. Free-Space Management

Bit Vector (Bitmap)

- One bit per block: **1 = free, 0 = allocated** (or vice versa).
- Simple and efficient to find contiguous free blocks.
- Size: (Disk size / Block size) bits.
- Example: 1TB disk, 4KB blocks → 256MB bitmap.

Linked List

- All free blocks linked together; first free block address stored.
- **Cannot find contiguous space easily**; poor performance.

Grouping

- First free block stores addresses of **n free blocks**.
- Last of those n stores addresses of next n, etc.
- Faster than linked list for finding free blocks.

Counting

- Store address of **first free block + count of contiguous free blocks** following it.
- Efficient when contiguous free blocks are common.

6. File Protection

Access Control

- Specify what operations a user can perform on a file.
- Common operations: **read (r), write (w), execute (x)**.

UNIX Permission Model

```
Owner: rwx  Group: r-x  Others: r--
```

- `-rwxr-xr--` = Owner can read/write/execute; Group can read/execute; Others can only read.

Access Control List (ACL)

- A **list associated with each file/directory** specifying access rights for each user or group.
- More flexible than UNIX permissions.
- Example: Windows NTFS uses ACLs.

Protection Mechanisms

- **Password protection** → File has a password.
 - **Capability list** → Each user/process has a list of permitted accesses.
 - **ACL** → Most common; fine-grained control.
-

7. I/O Systems

I/O Hardware

- **Port** → Connection point for a device.
- **Bus** → Set of wires; shared communication pathway (PCI, USB, SATA).
- **Controller** → Electronics that operate a port/bus/device (disk controller, USB controller).

I/O Controller

- Each device has a controller with **registers**: data-in, data-out, status, control.
- CPU communicates with controller by **reading/writing to these registers**.
- **Port-mapped I/O** → Uses special I/O instructions.
- **Memory-mapped I/O** → Registers mapped to memory addresses; CPU uses standard memory instructions.

Device Drivers

- OS module that provides a **standard interface** to a specific hardware device.
- Translates generic read/write calls to device-specific commands.
- Hides hardware complexity from the OS kernel.

Interrupt-Driven I/O

1. CPU issues I/O command to device controller.
2. CPU continues other work.
3. Device completes operation → sends **interrupt** to CPU.
4. CPU handles interrupt → reads result → continues process.

DMA (Direct Memory Access)

- For bulk data transfer, DMA controller handles data movement **between device and memory directly**.
- CPU sets up DMA (source, destination, size); DMA does the transfer; CPU interrupted only on completion.
- Dramatically improves efficiency for large I/O.

Buffering

- **Buffer** → Temporary memory area to store data while transferring between devices at different speeds.
- Types: Single buffer, Double buffer, Circular buffer.
- Helps match speed differences between devices.

Caching

- Keep a **copy of frequently used data in fast memory**.
- Difference from buffer: buffer is a temporary holding area; cache is a copy of existing data for speed.

Spooling

- **Simultaneous Peripheral Operations On-Line**.

- Used for devices that cannot interleave data (e.g., printer).
 - Jobs sent to a **spool (disk buffer)**; device processes them one at a time.
 - Example: Print spooler.
-

8. Disk Management

Disk Structure

Platter → Surface → Track → Sector → Block

- **Seek time** → Time to move disk arm to correct track.
- **Rotational latency** → Time for desired sector to rotate under head.
- **Transfer time** → Time to transfer data.
- **Disk access time = Seek + Rotational latency + Transfer time**

Disk Scheduling Goal

- Minimize **seek time** (dominant cost).
- Choose optimal order to service disk I/O requests.

Disk Formatting

- **Low-level formatting** → Divides disk into sectors; done by manufacturer.
- **Partitioning** → Divides disk into logical groups of cylinders.
- **High-level formatting** → Creates file system (boot block, free-space structure, empty directory).

Boot Block

- Contains **bootstrap program** loaded by BIOS/UEFI at startup.
- Located at a fixed location (usually block 0 / MBR).

Bad Blocks

- Sectors that are defective; cannot be used.
 - **Flagged** during formatting; OS and disk controller avoid them.
 - Modern disks automatically **remap** bad sectors to spare sectors.
-

9. Disk Scheduling Algorithms

Assume disk queue: 98, 183, 37, 122, 14, 124, 65, 67 | Head starts at **53** | Total cylinders: 0–199

FCFS — First Come First Served

- Service requests in **arrival order**.
- Fair but not efficient.

Order: 53→98→183→37→122→14→124→65→67

Total head movement = $|53-98|+|98-183|+|183-37|+\dots = \mathbf{640 \text{ cylinders}}$

SSTF — Shortest Seek Time First

- Service request **closest to current head position** next.
- Greedy; reduces average seek time.
- **Starvation** possible for distant requests.

From 53: 65→67→37→14→98→122→124→183 = **236 cylinders**

SCAN (Elevator Algorithm)

- Head moves in **one direction**, services all requests until it reaches end, then reverses.
- Efficient; no starvation.

From 53 (moving toward 0): 37→14→0→65→67→98→122→124→183 = **208 cylinders**

C-SCAN (Circular SCAN)

- Head moves in **one direction** only, services requests.

- On reaching end, **jumps back to beginning** (no servicing on return).
- More uniform wait times than SCAN.

From 53 (toward 199): 65→67→98→122→124→183→199→0→14→37 = **382 cylinders**
(including jump)

LOOK

- Like SCAN but head **only goes as far as last request** in each direction (doesn't go to disk end).

From 53 (toward 183): 65→67→98→122→124→183→37→14 = **208 cylinders**

C-LOOK (Circular LOOK)

- Like C-SCAN but only goes to **last request**, then jumps to **first request** on other end.

From 53: 65→67→98→122→124→183→14→37 = **322 cylinders**

Summary Table

Algorithm	Movement	Starvation	Notes
FCFS	High	No	Fair, simple
SSTF	Low–Med	Yes	Greedy
SCAN	Medium	No	Elevator
C-SCAN	Medium	No	Uniform wait
LOOK	Low–Med	No	Better than SCAN
C-LOOK	Low–Med	No	Best practical choice

C-LOOK is the best general-purpose disk scheduling algorithm.

UNIT 6 — LINUX, WINDOWS, AND MODERN OS

Linux

Linux History

- Created by **Linus Torvalds** in 1991 as a free, open-source UNIX-like OS.
- Based on **MINIX**; influenced by UNIX.
- Kernel licensed under **GPL (GNU General Public License)**.
- Powers servers, supercomputers, Android devices, and embedded systems.

Linux Architecture

```
User Space:  Applications → Shell → System Libraries (glibc)
                                     ↓
Kernel Space: System Call Interface
                ↓
                [Process Mgmt] [Memory Mgmt] [File Systems] [Networking] [Dev
                ↓
                Hardware (CPU, Memory, Disks, Network)
```

Linux Kernel

- **Monolithic kernel** with **modular design** (loadable kernel modules).
- Kernel runs in kernel mode; user programs run in user mode.
- Supports SMP (Symmetric Multiprocessing) — multiple CPUs.

Kernel Modules

- **Loadable Kernel Modules (LKM)** → Device drivers, file systems added at runtime without rebooting.

- Commands: `insmod` (insert), `rmmmod` (remove), `lsmod` (list).

Process Management in Linux

- Processes created via `fork()` and `exec()`.
- **task_struct** → Linux equivalent of PCB; contains all process info.
- Linux uses the term **task** for both processes and threads.
- `clone()` system call creates both processes and threads (threads share resources, processes don't).

Linux Scheduling

- **Completely Fair Scheduler (CFS)** → Default scheduler in Linux.
- Gives each process a fair share of CPU based on **virtual runtime (vruntime)**.
- Lower vruntime → higher priority for CPU.
- Uses a **red-black tree** to track runnable processes.
- **Priority:** Nice values from -20 (highest) to +19 (lowest).

Linux Memory Management

- Supports **paging** with 4-level page tables (x86-64).
- **Buddy system** for physical memory allocation (kernel).
- **Slab allocator** for kernel objects (cache frequently used objects).
- **mmap()** for memory-mapped files.
- Swap managed via swap partitions or swap files.

Linux File System

- Default: **ext4** (Fourth Extended File System).
- Also supports: Btrfs, XFS, FAT32, NTFS (via kernel modules), etc.
- **Virtual File System (VFS)** → Abstraction layer allowing different file systems to be accessed uniformly.
- **Inodes** → Data structure storing file metadata (not name).

- **Directory entries (dentry)** → Map file names to inodes.

Linux I/O

- Follows **Everything is a file** philosophy.
- Device files in `/dev` (e.g., `/dev/sda`, `/dev/null`).
- **Character devices** → byte-by-byte I/O (keyboard, serial).
- **Block devices** → block-level I/O (disk).
- I/O scheduling: **CFQ**, **Deadline**, **NOOP** schedulers.

Linux IPC

Mechanism	Description
Pipes	Unidirectional; parent-child communication
Named Pipes (FIFO)	Unrelated processes; persistent in filesystem
Signals	Asynchronous notifications (<code>kill</code> , <code>SIGTERM</code> , <code>SIGKILL</code>)
Shared Memory	Fastest; <code>shmget()</code> , <code>shmat()</code>
Message Queues	Structured messages; <code>msgget()</code> , <code>msgsnd()</code> , <code>msgrcv()</code>
Semaphores	Synchronization; <code>semget()</code> , <code>semop()</code>
Sockets	Network and local communication

Linux Networking

- **Network stack:** Application → Socket → TCP/UDP → IP → Network Driver → Hardware.
- Sockets used for all network communication.
- Commands: `ifconfig`, `ip`, `netstat`, `ss`.

Linux Security

- **DAC (Discretionary Access Control)** → UNIX permissions (owner, group, others).
 - **SELinux (Security-Enhanced Linux)** → MAC (Mandatory Access Control); policy-based.
 - **AppArmor** → Alternative MAC system.
 - **Capabilities** → Fine-grained privileges instead of binary root/non-root.
 - **Namespaces and cgroups** → Basis of containers (Docker, LXC).
-

Windows

Windows Architecture

```

User Mode:
Win32 Applications, POSIX, OS/2 Subsystems
↑ System Call Interface (NT API)
Kernel Mode:
Executive Services (I/O, Memory, Process, Security, IPC, PnP)
Kernel (scheduling, interrupt, exception handling)
HAL (Hardware Abstraction Layer)
Device Drivers
↑↓
Hardware
  
```

System Components

- **HAL (Hardware Abstraction Layer)** → Isolates OS from hardware differences; easy porting.
- **Kernel** → Handles scheduling, interrupts, exceptions; non-pageable.

- **Executive** → Collection of OS services: I/O Manager, Memory Manager, Process Manager, Security Reference Monitor, Object Manager, PnP Manager, Power Manager.
- **Win32 Subsystem** → Provides Windows API to user applications.
- **NTDLL.DLL** → Bridge between user-mode and kernel-mode (system call stubs).

Process Management in Windows

- **Process** → Container for threads; has virtual address space, handles, and security token.
- **Thread** → Unit of execution.
- Each process has at least one thread.
- **Fibers** → User-mode threads scheduled by application.
- **Jobs** → Group of processes managed as a unit.
- Priority: 0 (idle) to 31 (real-time); dynamic priorities for normal threads.

Memory Management in Windows

- Uses **demand paging** with **clustering** (bring extra pages to reduce page faults).
- **Virtual Memory Manager (VMM)** manages physical memory.
- 4KB pages (standard); 2MB large pages supported.
- **Working Set** → Set of pages currently in physical memory for a process.
- Memory mapped files via `CreateFileMapping()` and `MapViewOfFile()`.
- **Heap Manager** → For dynamic memory allocation (`HeapAlloc`, `VirtualAlloc`).

File System in Windows

- **NTFS (New Technology File System)** → Default; supports ACLs, journaling, compression, encryption.
- Also: FAT32, exFAT, ReFS (Resilient File System).
- **Journaling** → NTFS logs changes before committing; crash recovery.
- **Alternate Data Streams (ADS)** → Multiple data streams per file.
- Mount points and drive letters (C:\, D:\, etc.).

Networking in Windows

- **Winsock (Windows Sockets)** → API for network programming; compatible with BSD sockets.
- **Named Pipes** → IPC over network.
- **RPC (Remote Procedure Call)** → Calling functions on remote machines.
- Active Directory for network authentication.

Security in Windows

- **SID (Security Identifier)** → Unique ID for users and groups.
- **Access Token** → Given to each process; contains SID and privileges.
- **ACL (Access Control List)** → DACL (what access is allowed) + SACL (auditing).
- **UAC (User Account Control)** → Prompts for elevation when admin rights needed.
- **BitLocker** → Full-disk encryption.
- **Windows Defender** → Built-in antivirus.

Modern OS Concepts

Android OS

- Mobile OS developed by **Google**; based on **Linux kernel**.

Architecture layers: 1. **Linux Kernel** → Hardware drivers, memory, power management. 2. **Hardware Abstraction Layer (HAL)** → Standard interface for hardware. 3. **Android Runtime (ART)** → Executes .dex (Dalvik Executable) bytecode; ahead-of-time compilation. 4. **Native Libraries** → C/C++ libraries (OpenGL, SQLite, WebKit). 5. **Application Framework** → Java APIs for app development (Activity Manager, Content Providers). 6. **Applications** → System apps + user-installed apps.

Binder IPC → Android's main IPC mechanism (fast, secure).

- **Zygote** → Pre-forked process from which all apps are spawned.
- **Permissions model** → Apps declare permissions; user grants at install or runtime.

Mobile Operating Systems

- Designed for **resource-constrained** devices (limited battery, CPU, memory).
- Key features: touch input, GPS, accelerometer, power efficiency.
- **iOS** → Apple's mobile OS; based on Darwin (BSD + Mach microkernel); closed ecosystem.
- **Android** → Google's mobile OS; open source; largest market share.
- **Mobile scheduling** → Battery-aware; background apps throttled or suspended.
- **Mobile memory** → No traditional swap; apps killed when memory is low.

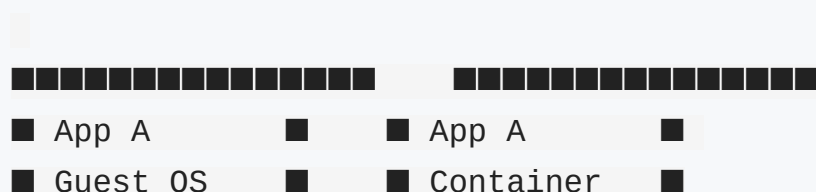
Virtual Machines

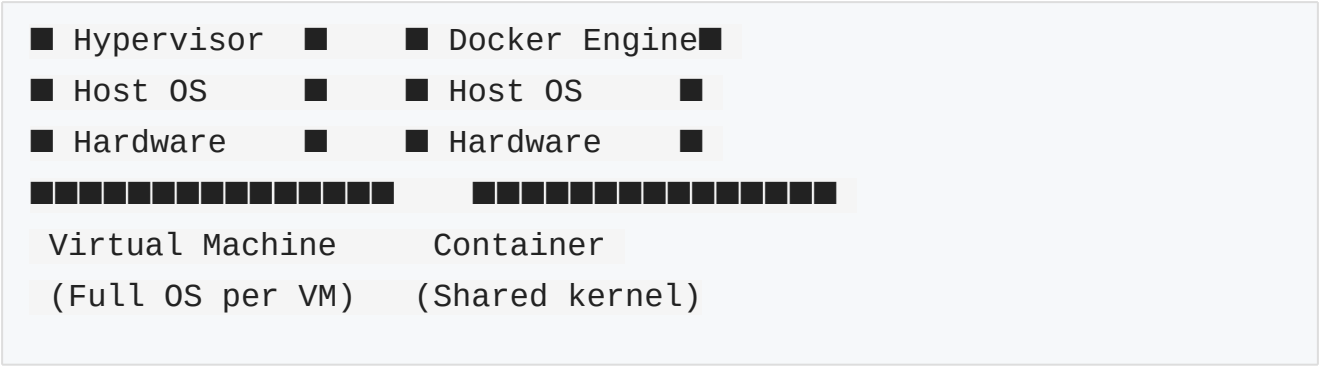
- **VM (Virtual Machine)** → Software emulation of a complete computer system.
- Each VM runs its own OS and applications.
- **Hypervisor (VMM)** → Manages VMs.
- **Type 1** → Bare metal: VMware ESXi, Hyper-V, Xen.
- **Type 2** → Hosted: VirtualBox, VMware Workstation.
- Uses: Server consolidation, testing, cloud computing, isolation.

Containers

- **Container** → A lightweight, isolated environment for running applications.
- Shares the **host OS kernel** (unlike VMs which have their own kernel).
- Faster startup, smaller size, less overhead than VMs.
- **Docker** → Most popular container platform.
- **LXC (Linux Containers)** → OS-level virtualization.
- Key technologies: **Namespaces** (isolation), **cgroups** (resource limits), **Union File System** (layered filesystem).

VMs vs Containers:





Cloud Operating Systems

- An **OS paradigm** where resources are provisioned over the internet.
- Traditional OS manages one machine; cloud OS manages **distributed resources**.
- Key concepts:
- **IaaS (Infrastructure as a Service)** → Virtual machines on demand (AWS EC2, Azure VM).
- **PaaS (Platform as a Service)** → Managed runtime environment (Google App Engine).
- **SaaS (Software as a Service)** → Ready-to-use applications (Gmail, Office 365).
- **Kubernetes** → Container orchestration system; manages clusters of containers.
- **OpenStack** → Open-source cloud OS for managing large pools of compute, storage, and networking.
- Characteristics: Elasticity, pay-per-use, multi-tenancy, high availability.

Quick Revision Summary

Unit	Key Topics
Unit 1	OS types, OS structure (monolithic/micro/modular), system calls, boot, virtualization

Unit 2	Process states, PCB, fork/exec/wait, scheduling algorithms (FCFS/SJF/RR), threads, IPC
Unit 3	Critical section, Peterson's, semaphores, Producer-Consumer, Deadlock (4 conditions), Banker's algorithm
Unit 4	Paging, segmentation, virtual memory, demand paging, page replacement (FIFO/OPT/LRU), thrashing
Unit 5	File allocation (contiguous/linked/indexed), disk scheduling (SSTF/SCAN/C-LOOK), I/O management
Unit 6	Linux (CFS, ext4, VFS), Windows (NTFS, HAL, ACL), Android, containers vs VMs, cloud OS

Notes prepared for: College Exams | Placement Preparation | Technical Interviews | Quick Revision